Introduction

Motivation I

Modern research software repositories increasingly adopt monorepo designs in which multiple projects share a common set of curated resources. A monorepo consolidates source code, documentation, datasets, and governance artefacts under one version-controlled root, enabling atomic cross-project changes and a single source of truth for shared data [@Fowler2002patterns]. The practical benefit is significant: a bibliography updated once in `fonds/templates/template_bibliography/` is immediately available to every project that discovers it at runtime, without any per-project copy.

Three categories of shared resource appear consistently across research template repositories:

1. **Data pools (fonds)**: curated reference sets — bibliographies, contact registries, dataset catalogues — that projects query but must never mutate.

Motivation II

2. **Governance rules**: machine-readable constraint schemas and human-readable style guidelines that projects load to validate their own outputs.
3. **Executable tools**: script-based entry points that projects invoke to run computations, validate artefacts, or call external agents.

Without a canonical integration pattern for consuming these resources, projects face a dilemma: they can hard-code discovery paths (creating fragile, repo-root-sensitive logic) or skip resource consumption entirely (forfeiting the monorepo's collaborative benefits). Neither outcome is acceptable in a public, forkable template repository intended to demonstrate best practices [@Wilson2014best].

Contribution I

This paper introduces `template_pools_rules_tools`, a **meta-project exemplar** that resolves this dilemma with a four-module architecture (@fig:architecture). Each module handles one resource category plus a fourth orchestration module:

Module	Resource category	Key function
`src/fonds_reader.py`	Data pools	`read_all_fonds()`
`src/rules_applier.py`	Governance rules	`validate_against_rules()`
`src/tools_invoker.py`	Executable tools	`discover_tools()`
`src/integration.py`	All three	`run_integration_demo()`

Contribution II

The architecture obeys three design invariants:

- **Read-only resource access**: no module writes to `fonds/`, `rules/`, or `tools/`. The Layer Contract in `AGENTS.md` enforces this at code-review time.
- **Repo-root-relative discovery**: all path resolution uses `pathlib.Path(__file__).resolve().parents[N]` so that scripts work from any working directory.
- **Graceful degradation**: every reader catches `FileNotFoundError` and `yaml.YAMLError`, logs a warning, and returns a safe empty value. The pipeline never raises on a missing resource.

Paper Organisation I

The remainder of this paper is structured as follows. @sec:pools describes the fond layer and the `fonds_reader` module. @sec:rules describes the rules layer and the `rules_applier` module. @sec:tools describes the tool layer and the `tools_invoker` module. @sec:integration presents the unified integration pipeline, the manuscript variable token system, and a discussion of resilience design. @sec:conclusion summarises key findings and future directions.

The architecture overview in @fig:architecture provides a visual map of these relationships. Runtime statistics collected during integration are visualised in @fig:counts.